龙芯 2K300/301 系列板卡使用手册-GPIO 输入输出使用

龙芯 2K300/301 开源库使用说明

GPIO 输入输出篇

一、GPIO 模块简介

GPIO（General Purpose Input/Output，通用输入输出）是龙芯 2K300/301 处理器上最基础的硬件外设之一。通过 GPIO 引脚，可以实现数字信号的输入输出控制，适用于 LED 控制、按键检测、传感器数据读取等应用场景。

本库提供了完整的 GPIO 驱动封装，支持线程安全的 GPIO 操作。

二、包含头文件

在使用 GPIO 功能之前，需要在代码中包含以下头文件：

```c++
#include "lq_reg_gpio.hpp"
```

该头文件定义了 GPIO 相关的类、枚举类型和函数。

三、数据类型定义

1. GPIO 引脚枚举 (gpio_pin_t)

用于指定使用的 GPIO 引脚号：

PIN_0 ~ PIN_15 (0x00 ~ 0x0F)

PIN_16 ~ PIN_31 (0x10 ~ 0x1F)

PIN_32 ~ PIN_47 (0x20 ~ 0x2F)

PIN_48 ~ PIN_63 (0x30 ~ 0x3F)

PIN_64 ~ PIN_79 (0x40 ~ 0x4F)

PIN_80 ~ PIN_95 (0x50 ~ 0x5F)

PIN_96 ~ PIN_105 (0x60 ~ 0x69)

示例：PIN_1, PIN_88, PIN_44 等

2. GPIO 模式枚举 (gpio_mode_t)

GPIO_MODE_OUT 输出模式

GPIO_MODE_IN 输入模式

3. GPIO 电平枚举 (gpio_level_t)

GPIO_LOW 低电平（逻辑 0）

GPIO_HIGH 高电平（逻辑 1）

4. GPIO 复用模式枚举 (gpio_mux_mode_t)

GPIO_MUX_GPIO GPIO 复用（默认）

GPIO_MUX_ALT1 第一复用

GPIO_MUX_ALT2 第二复用

GPIO_MUX_MAIN 主复用

四、ls_gpio 类接口说明

类名：ls_gpio

构造函数：

`ls_gpio()`

无参构造函数，创建一个未初始化的 GPIO 对象

`ls_gpio(gpio_pin_t _gpio, gpio_mode_t _mode, gpio_mux_mode_t _mux = GPIO_MUX_GPIO)`

有参构造函数，初始化 GPIO 对象

参数：

_gpio - GPIO 引脚号

_mode - GPIO 模式（输入/输出）

_mux - GPIO 复用模式（默认为 GPIO 功能）

成员函数：

`void gpio_direction_set(gpio_mode_t _mode)`

设置 GPIO 的输入输出模式

参数：_mode - GPIO 模式

`void gpio_level_set(gpio_level_t value)`

设置 GPIO 输出电平（仅输出模式有效）

参数：value - 输出电平（GPIO_LOW 或 GPIO_HIGH）

`gpio_level_t gpio_level_get()`

获取 GPIO 输入电平（仅输入模式有效）

返回：当前引脚电平状态

`gpio_pin_t gpio_get_pin()`

获取当前 GPIO 的引脚号

`gpio_mode_t gpio_get_mode()`

获取当前 GPIO 的模式

`gpio_mux_mode_t gpio_get_mux()`

获取当前 GPIO 的复用模式

五、GPIO 输出示例（LED 控制）

【示例 1】控制核心板三色灯（流水灯）

```c++
#include "lq_reg_gpio.hpp"

int main()
{
    // 初始化核心板上的三色灯为输出模式
    ls_gpio GREEN(PIN_1, GPIO_MODE_OUT);
    ls_gpio RED(PIN_2, GPIO_MODE_OUT);
    ls_gpio BLUE(PIN_3, GPIO_MODE_OUT);

    while (1)
    {
        // 绿灯亮，红灯灭，蓝灯灭
        GREEN.gpio_level_set(GPIO_LOW);
```

```
        RED.gpio_level_set(GPIO_HIGH);
        BLUE.gpio_level_set(GPIO_HIGH);
        usleep(500000);  // 延时 500ms

        // 绿灯灭，红灯亮，蓝灯灭
        GREEN.gpio_level_set(GPIO_HIGH);
        RED.gpio_level_set(GPIO_LOW);
        BLUE.gpio_level_set(GPIO_HIGH);
        usleep(500000);

        // 绿灯灭，红灯灭，蓝灯亮
        GREEN.gpio_level_set(GPIO_HIGH);
        RED.gpio_level_set(GPIO_HIGH);
        BLUE.gpio_level_set(GPIO_LOW);
        usleep(500000);
    }

    return 0;
}
```

【示例 2】控制蜂鸣器

```c++
#include "lq_reg_gpio.hpp"

int main()
{
    // 初始化蜂鸣器引脚为输出模式（PIN_12）
    ls_gpio BEEPER(PIN_12, GPIO_MODE_OUT);

    while (1)
    {
        BEEPER.gpio_level_set(GPIO_LOW);   // 蜂鸣器鸣叫
        usleep(500000);                    // 延时 500ms
        BEEPER.gpio_level_set(GPIO_HIGH);  // 蜂鸣器停止
        usleep(500000);
    }

    return 0;
}
```

六、GPIO 输入示例（按键检测）

【示例 1】读取多个按键和拨码开关

```
c++
#include "lq_reg_gpio.hpp"
#include <stdio.h>
#include <unistd.h>

int main()
{
    // 初始化母板上的按键与拨码开关
    ls_gpio gpio44(PIN_44, GPIO_MODE_IN);
    ls_gpio gpio45(PIN_45, GPIO_MODE_IN);
    ls_gpio gpio80(PIN_80, GPIO_MODE_IN);
    ls_gpio gpio17(PIN_17, GPIO_MODE_IN);
    ls_gpio gpio20(PIN_20, GPIO_MODE_IN);

    while (1)
    {
        // 读取并打印各引脚的高低电平状态
        printf("PIN_44: %d, PIN_45: %d, PIN_80: %d, PIN_17: %d,
PIN_20: %d\n",
               gpio44.gpio_level_get(),
               gpio45.gpio_level_get(),
               gpio80.gpio_level_get(),
               gpio17.gpio_level_get(),
               gpio20.gpio_level_get());

        usleep(500000);  // 500ms 延时
    }

    return 0;
}
```

七、GPIO 进阶用法

【示例 1】GPIO 对象的拷贝和赋值

```
c++
```

```
#include "lq_reg_gpio.hpp"

int main()
{
    // 创建一个 GPIO 对象
    ls_gpio gpio1(PIN_88, GPIO_MODE_OUT);

    // 拷贝构造（共享底层资源）
    ls_gpio gpio2(gpio1);

    // 拷贝赋值（共享底层资源）
    ls_gpio gpio3 = gpio1;

    // 三个对象控制同一个引脚
    gpio1.gpio_level_set(GPIO_HIGH);
    gpio2.gpio_level_set(GPIO_LOW);
    gpio3.gpio_level_set(GPIO_HIGH);

    return 0;
}
```

八、注意事项

1. 引脚配置

- 使用 GPIO 前必须先了解硬件原理图，确认引脚号和功能
- 不同引脚可能有不同的电气特性（上拉、下拉等）
- 避免将输出引脚直接短路

2. 电平逻辑

- GPIO_LOW 表示逻辑 0，通常是低电平（0V）
- GPIO_HIGH 表示逻辑 1，通常是高电平（3.3V）
- 具体电平范围需参考硬件规格书

3. 资源管理

- GPIO 对象析构时会自动释放资源
- 使用智能指针管理 GPIO 资源可以避免资源泄漏

九、常见问题

Q1: 如何确定使用哪个引脚?

A1: 请查看硬件原理图和引脚定义文档，确认每个引脚的功能和连接。

Q2: GPIO 读取的值不对怎么办?

A2: 检查以下几点:

- 引脚是否正确配置为输入模式
- 引脚是否有外部电路干扰
- 是否有上拉或下拉电阻
- 引脚是否被复用为其他功能

Q3: 输出电平不稳定?

A3: 可能是以下原因:

- 输出负载过大
- 引脚有外部强信号干扰
- 电源供电不稳定
- 引脚配置错误

Q4: 可以同时操作多个 GPIO 吗?

A4: 可以，但需要注意:

- 创建多个 GPIO 对象
- 确保每个对象控制不同的引脚
- 在多线程中操作是安全的

Q5: 如何获取 GPIO 的当前状态?

A5: 使用 gpio_get_pin()、gpio_get_mode()、gpio_get_mux() 函数，可以获取 GPIO 的引脚号、模式和复用模式。

十、参考资源

- 龙芯 2K300/301 处理器用户手册
- 龙芯 2K300/301 开发板原理图
- GPIO 示例代码：example/src/01_lq_gpio_basic_demo.cpp
- GPIO 驱动代码:

- libraries/drv/inc/lq_reg_gpio.hpp
- libraries/drv/src/lq_reg_gpio.cpp